

Silent Failure Detector

A monitoring system for silent data pipeline failures

Pipelines fail in two ways. **Loudly** — you get an error and fix it. **Silently** — the pipeline completes successfully, dashboards stay green, but the underlying data is wrong and nobody knows. This project catches the second kind.

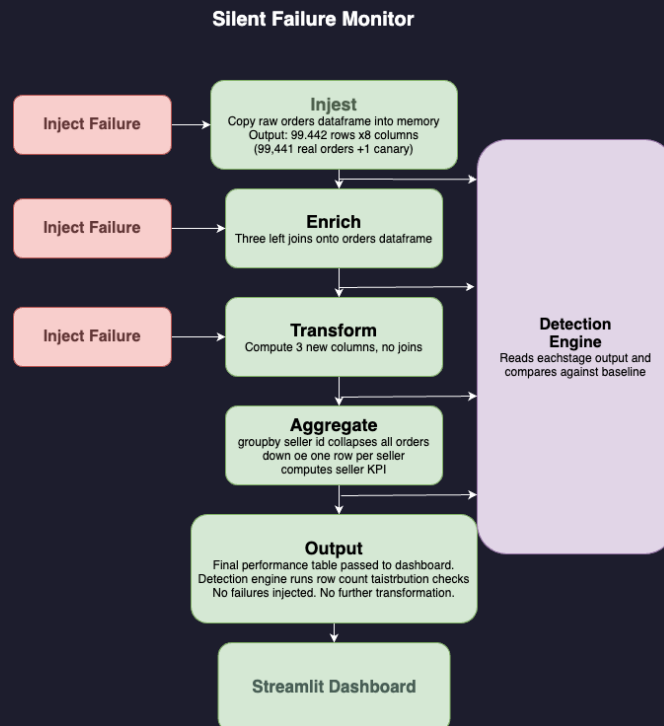
Built on the real Olist Brazilian e-commerce dataset — 99,441 orders, 2016–2018.

Architecture

The system loads the Olist CSVs once and runs the data through a 5-stage pipeline twice — once clean to establish a baseline, once with a failure injected. The detection engine compares the two across every column at every stage. The pipeline always completes without errors regardless of what is injected.

Pipeline Stages

Each stage is a Python function that takes a dataframe in and returns a dataframe out. The data gets progressively richer as it flows through. A failure injected at stage 1 quietly propagates and distorts every subsequent stage.



Stage 1 — Ingest

The simplest stage. We take the raw orders table and return it. In a real pipeline this is where you would read from S3 or a database. Here we copy the orders dataframe.

This is where **record drops**, **stale timestamps**, and **canary removal** are injected — because these are the kinds of things that go wrong at the source, before any processing has happened.

Stage 2 — Enrich

This is where we join everything together. We take the orders from stage 1 and attach customer details, order items, and payment info onto each row. After this stage, each row represents one order with everything we know

about it — who bought it, what they bought, how they paid, how much it cost.

This is where **null floods** and **duplicate injections** happen — because these are join-related failures. A bad join produces nulls, a repeated join produces duplicates.

Stage 3 — Transform

We do not change the shape of the data here — same rows, same join keys. Instead we calculate three new columns from existing ones:

- `delivery_days` — how many days from purchase to delivery
- `is_late` — whether the order arrived after the estimated delivery date
- `total_order_value` — price plus freight value

This is where **schema drift** is injected — because if price got silently cast to string in stage 2, the addition `price + freight_value` here will silently produce NaN instead of crashing.

Stage 4 — Aggregate

This is the final transformation — we collapse everything down to one row per seller. All the order-level detail gets summarised into KPIs:

- `total_orders` — how many orders this seller fulfilled
- `total_revenue` — sum of all order values
- `avg_delivery_days` — average time from purchase to delivery
- `late_delivery_rate` — fraction of orders that arrived late
- `avg_review_score` — average customer review score

This is the stage that matters most to the business. If anything went wrong upstream — dropped records, corrupted prices, duplicate items — it all shows up here as wrong numbers. But the table looks fine. No errors, no nulls, just wrong KPIs.

Failure Types

Each failure is injected in memory on the dataframes after loading. The CSV files are never modified. The pipeline never raises an exception.

Failure	Stage	What It Simulates
Record Drop	1 — Ingest	Rows silently removed — broken join or partial read
Stale Timestamps	1 — Ingest	All timestamps frozen — cache stopped updating
Canary Removal	1 — Ingest	Sentinel record deleted — filter bug eating specific rows
Null Flood	2 — Enrich	Column fills with NaN — incomplete upstream source
Duplicate Records	2 — Enrich	Rows duplicated — inflates counts and revenue
Price Corruption	2 — Enrich	Prices set to 0 or negative — currency conversion bug
Schema Drift	3 — Transform	Price column cast to string — upstream format change

Dashboard Guide

The Sidebar — Failure Injection

This is where you control what goes wrong. Each toggle injects a specific silent failure into the pipeline when you hit Run. If nothing is toggled, the pipeline runs clean. The sliders under each toggle control the intensity — how many rows to drop, how high the null rate goes.

How to Tell If the Pipeline Ran Successfully

The pipeline itself always runs successfully — that is the whole point of silent failures. You will never see a crash or an error message from the pipeline itself. What tells you the data is healthy is the detector. You want to see:

- Overall Health at 100% or close to it
- Zero **criticals**
- Zero or very few warnings
- The green banner: "No failures injected — clean run baseline"

If you see **criticals**, the pipeline completed but the data coming out of it is wrong.

Top Metrics Row

Five numbers giving you a first-glance summary after a run:

- **Overall Health %** — what percentage of all checks across all stages passed. 100% means nothing suspicious was found.
- **Checks Run** — total number of individual checks that executed across all stages.
- **Passed** — how many checks returned passed.
- **Critical** — definitive failures. Act on these immediately.
- **Warnings** — suspicious but could have an innocent explanation. Investigate.



Stage Health Bar Chart

A horizontal bar for each stage showing its health percentage. Green means all checks passed. Orange means some warnings. Red means critical failures. This tells you where in the pipeline something went wrong — knowing which stage is the starting point for investigation.

When you inject a record drop, you will see stage 1 go red immediately because that is where the drop happens. Every subsequent stage also shows degraded health because the dropped records propagate forward.

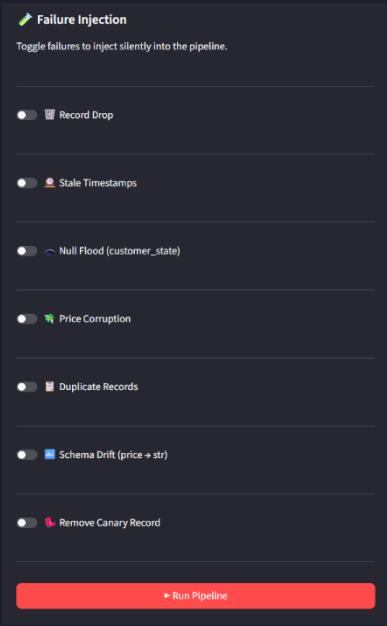
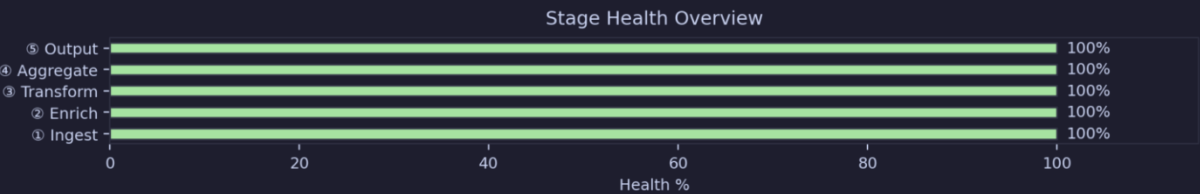


Figure 1: The Sidebar Control

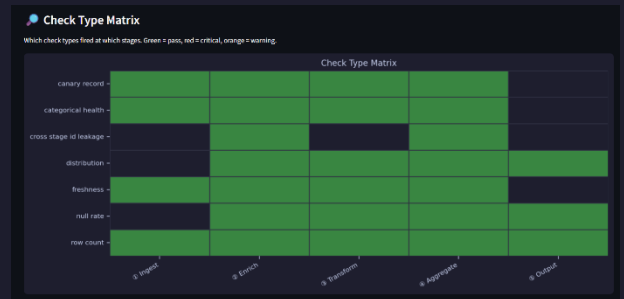
Row Counts Chart

Two bars side by side for each stage — blue for baseline, red for the current run. A clean run shows both bars at identical height. Inject a 25% record drop and the red bars shrink noticeably from stage 1 onward. Inject duplicates and the red bars spike above the blue at the enrich stage where items get duplicated.

The chart also shows something important about the pipeline structure — the enrich stage naturally has more rows than ingest because of the joins. That is normal and expected.

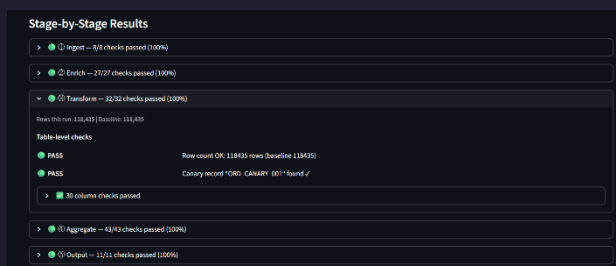
Check Type Matrix

A heatmap where rows are check types and columns are stages. Green cells passed, orange is a warning, red is critical. This gives you the full picture in one view — which specific check failed at which specific stage. For example if you inject null flood, the null rate cell goes orange at stage 2 and stays orange at stages 3 and 4 because the nulls propagate forward.



Column Health Matrix

A heatmap where rows are dataset columns and columns are pipeline stages. This is where you trace exactly which column first becomes unhealthy and how far the corruption travels downstream. Grey cells mean the column does not exist at that stage yet.



Stage-by-Stage Detail

The most detailed section. Each stage is an expandable panel showing every individual check with its badge, message, and expected vs actual values. This is where you read the human explanation of what went wrong — not just "something failed at stage 2" but "null rate for customer_state is 42.3% when we expected less than 5%." That is actionable information.

Run History Table

Every time you hit Run, a row gets added showing the timestamp, how many failures were injected, the overall health, and the critical and warning counts. This lets you compare runs — inject one failure, note the health score, add another, see how the score changes.

Raw Stage Data Explorer

A dropdown where you can select any stage and browse the actual dataframe up to 100 rows. This is your ground truth. If the detector flags negative prices at stage 3, you can come here, select stage 3, and see the -99.99 values in the price column with your own eyes.

A Concrete Example

Say you toggle Record Drop at 25% and hit Run:

- The red banner tells you failures were injected silently
- Overall health drops — around 60%
- The stage health chart shows stage 1 in red
- The row count chart shows all red bars significantly shorter than blue
- The check matrix shows row_count_drop cells red across multiple stages

- Stage-by-stage detail shows "Row count dropped 22.6% from baseline" at stage 1 with cascading warnings downstream
- The raw data explorer at stage 1 shows noticeably fewer rows than the baseline

The pipeline reported no errors. Everything completed. But the dashboard caught it.

Known Limitations

1. **Single dataset baseline.** The baseline is one clean run of the same dataset, not a rolling historical average. The detector compares the pipeline against itself rather than against genuine historical variation.
2. **Fixed thresholds.** All detection thresholds are hardcoded in THRESHOLDS.md. They do not adapt to natural seasonal variation in the data over time.
3. **No time dimension.** Every run is isolated. The run history table accumulates within a single session only and resets when you restart Streamlit.

Version 2 — Coming

Version 2 addresses all three limitations by introducing a time dimension and a learned baseline:

Component	What It Does
metadata_store.py	Saves a metadata summary row after every pipeline run
Time simulation	Slices Olist data by date — simulates 60 or more daily runs
baseline_engine.py	Computes rolling mean and standard deviation from run history
Dynamic thresholds	Mean plus or minus N standard deviations — no hardcoded values
ML anomaly detection	Learns what normal looks like without any manual configuration

Version 2 is the system described in the original project brief. Version 1 is the foundation that makes Version 2 interpretable.